week_9\week9\src\main\java\week9\Board2D.java

```java
package week9;
import java.io.*;

/**
 * The Board2D class represents a 2D tic-tac-toe board.
 * It provides methods for printing the board, checking if the board is full, checking if a cell is empty,
 * getting the position of a cell based on its number, placing a marker on the board, and checking for a winner.
 * The class uses a 2D array to represent the board and implements the necessary logic to determine the game state.
 */
public class Board2D extends Board {
    private char board[][];
    private PrintStream out;

    /**
     * Constructor for Board2D class.
     * Initializes the board as a 2D array of characters and fills it with '0' to represent empty cells.
     * @param out
     */
    public Board2D(PrintStream out){
        this.out = out;
        board = new char[Constants.ROW][Constants.COL];
        for(int i = 0; i < Constants.ROW; i++){
            for(int j = 0; j < Constants.COL; j++){
                board[i][j] = '0';
            }
        }
    }

    /**
     * Prints the current state of the board.
     */
    @Override
    public void print(){
        for(int i = 0; i < Constants.ROW; i++){
            for(int j = 0; j < Constants.COL; j++){
                out.printf("| %s ", "" + board[i][j]);
            }
            out.println("|");
        }
    }

    /**
     * Returns the total number of cells on the board.
     * @return the total number of cells
     */
    @Override
    public int getTotalCells(){
        return Constants.ROW * Constants.COL;
```

```
49      }
50
51      /**
52       * Checks if the board is full.
53       * @return true if the board is full, false otherwise
54       */
55      @Override
56      public boolean isBoardFull(){
57          for(int i = 0; i < Constants.ROW; i++){
58              for(int j = 0; j < Constants.COL; j++){
59                  if(board[i][j] == '0'){
60                      return false;
61                  }
62              }
63          }
64          return true;
65      }
66
67      /**
68       * Checks if a cell is empty.
69       * @param pos the position of the cell to check
70       * @return true if the cell is empty, false otherwise
71       */
72      @Override
73      public boolean isCellEmpty(Position pos){
74          return board[pos.getRow()][pos.getCol()] == '0';
75      }
76
77      /**
78       * Gets the position of a cell based on its box number.
79       * The box number is a 1-based index that corresponds to the cell's position
    on the board.
80       * @param boxNumber the number of the cell (1-based index)
81       * @return the Position object representing the cell's position on the board
82       */
83      @Override
84      public Position getCellPosition(int boxNumber){
85          int row = (boxNumber - 1) / Constants.COL;
86          int col = (boxNumber - 1) % Constants.COL;
87          return new Position(row, col);
88      }
89
90      /**
91       * Places a marker on the board at the specified position.
92       * @param pos the position where the marker should be placed
93       * @param marker the marker to place on the board
94       */
95      @Override
96      public void placeMarker(Position pos, char marker){
97          board[pos.getRow()][pos.getCol()] = marker;
98      }
99
100     /**
101      * Checks if there is a winner on the board.
```

```
     * @return the marker of the winning player, or '0' if there is no winner
     */
    @Override
    public char checkWinner(){
        // Left → Right
        for(int i = 0; i < Constants.ROW; i++){
            for(int j = 0; j ⩽ Constants.COL - Constants.WIN_LENGTH; j++){
                if(checkLine(new Position(i, j), 0, 1)){
                    return board[i][j];
                }
            }
        }
        // Top → Bottom
        for(int j = 0; j < Constants.COL; j++){
            for(int i = 0; i ⩽ Constants.ROW - Constants.WIN_LENGTH; i++){
                if(checkLine(new Position(i, j), 1, 0)){
                    return board[i][j];
                }
            }
        }

        // Diagonal
        for(int i = 0; i ⩽ Constants.ROW - Constants.WIN_LENGTH; i++){
            for(int j = 0; j ⩽ Constants.COL - Constants.WIN_LENGTH; j++){
                if(checkLine(new Position(i, j), 1, 1)){
                    return board[i][j];
                }
            }
        }

        // Anti-diagonal
        for(int i = 0; i ⩽ Constants.ROW - Constants.WIN_LENGTH; i++){
            for(int j = Constants.WIN_LENGTH - 1; j < Constants.COL; j++){
                if(checkLine(new Position(i, j), 1, -1)){
                    return board[i][j];
                }
            }
        }
        return '0';
    }

    /**
     * Checks if a line has the same marker.
     * @param startPos the starting position of the line
     * @param steps the steps to move in each direction
     * @return true if the line has the same marker, false otherwise
     */
    @Override
    protected boolean checkLine(Position startPos, int... steps){
        int startRow = startPos.getRow();
        int startCol = startPos.getCol();
        int rowStep = steps[0];
        int colStep = steps[1];
        char marker = board[startRow][startCol];
```

```
156         if(marker == '0'){
157             return false;
158         }
159 
160         for(int k = 1; k < Constants.WIN_LENGTH; k++){
161             int row = startRow + k * rowStep;
162             int col = startCol + k * colStep;
163 
164             // Out of Bound
165             if(row < 0 || row ⩾ Constants.ROW || col < 0 || col ⩾ Constants.COL)
{
166                 return false;
167             }
168 
169             // Not the same marker
170             if(board[row][col] ≠ marker){
171                 return false;
172             }
173         }
174         return true;
175     }
176 
177     /**
178      * Take the board and make it into a string for transportation in a network
179      * Example, which I hope will look like this: 000010002
180      * @return a string of characters represent each cell's state in order from
box 1 - the final cell
181      * which is 9 btw ;)
182      */
183     public String networkString(){
184         StringBuilder stringBuilder = new StringBuilder();
185 
186         for(int i = 1; i ⩽ getTotalCells(); i++){
187             Position position = getCellPosition(i);
188             char marker = board[position.getRow()][position.getCol()];
189             stringBuilder.append(marker);
190         }
191         return stringBuilder.toString();
192     }
193 
194     /**
195      * The reverse of networkString method, this methid take the string from the
network
196      * and then rebuild it
197      * @param string a 9-character-ish (can be more but well this is a 3 x 3
board)
198      * containing the marker for boxes 1-9
199      */
200     public void loadString(String string){
201         for (int i = 0; i < string.length(); i++){
202             char marker = string.charAt(i);
203             int boxNumber = i + 1;
204             Position position = getCellPosition(boxNumber);
205 
```

```
206                 placeMarker(position, marker);
207             }
208         }
209     }
210 
```